Mission 3 : Projet « Consultation des formations

Le CROSL (Comité Régional olympique et Sportif de Lorraine)

Sommaire

1. Introduction **2**
1.1 Contexte et Objectif du Projet 2
1.2 Technologies Utilisées 2
1.3 Architecture Globale du Système 2
2. La Base de Données (Backend) **3**
2.1 Choix technologiques et Hébergement 3
3. L'API Web (Le Serveur PHP) **4**
3.1 Rôle et Architecture de l'API 4
3.2 Sécurité et Connexion aux données (PDO) 4
3.3 Traitement et Formatage : Exemple d'un Endpoint 5
4. L'Application Android (Frontend) **6**
4.1 Architecture du Projet et Interface Utilisateur (UI) 6
4.2 Communication Réseau et Asynchronisme 6
4.3 Traitement des données et Affichage Dynamique (RecyclerView) 7
4.4 Navigation et Transmission de Données (Intents) 7
5. Limites du projet et Axes d'amélioration **8**
5.1 Dépendance totale au réseau (Pas de mode hors-ligne) 8
5.2 Sécurité des communications (HTTP) 8
Conclusion **8**
Annexe **9**
1. Aperçu de l'Interface Utilisateur 9

1. Introduction

1.1 Contexte et Objectif du Projet

L'application **App Forma** a été conçue pour répondre aux besoins du CROSL (Comité Régional Olympique et Sportif). Son objectif principal est de fournir aux utilisateurs un catalogue mobile interactif et dynamique permettant de consulter les différentes formations proposées, classées par domaines.

Voir l'annexe

Afin de garantir une gestion centralisée et en temps réel des données, le projet repose sur une architecture de type **Client-Serveur**.

1.2 Technologies Utilisées

Pour mener à bien ce projet, j'ai divisé le développement en deux grands pôles (Frontend et Backend) en utilisant les technologies suivantes :

- **Frontend (L'application Mobile) :**
  - **Langage :** Kotlin (Logique) et XML (Interface utilisateur).
  - **Environnement :** Android Studio.
  - **Composants clés :** `RecyclerView` pour l'affichage dynamique des listes, et exécution de tâches asynchrones (`Thread`) pour les appels réseau.
- **Backend (Le Serveur et l'API) :**
  - **Serveur Local :** WAMP (Apache).
  - **Langage de l'API :** PHP 8 (Création d'une API de type RESTful).
  - **Base de Données :** MySQL (gérée via phpMyAdmin).
- **Protocole d'échange :** Les données transitent entre le serveur et l'application mobile au format standard **JSON** via des requêtes HTTP.

1.3 Architecture Globale du Système

Le fonctionnement de l'application repose sur le principe suivant : L'application Android (le *Client*) ne contient aucune donnée en dur. Lorsqu'un utilisateur navigue sur l'application, celle-ci envoie une requête HTTP (via l'adresse locale `10.0.2.2` sur l'émulateur) vers les scripts PHP hébergés sur le serveur Apache. Le script PHP se connecte à la base de données MySQL via **PDO** (PHP Data Objects), récupère les informations demandées de manière sécurisée (requêtes préparées), et les renvoie à l'application mobile sous forme de tableau JSON. L'application se charge ensuite de traduire ce JSON pour l'afficher visuellement à l'utilisateur.

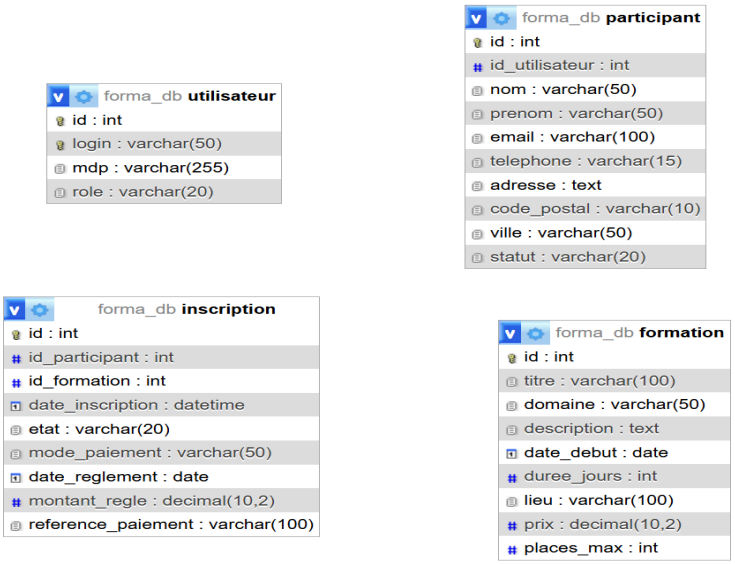
2. La Base de Données (Backend)

2.1 Choix technologiques et Hébergement

Le stockage des informations de l'application AppForma repose sur un Système de Gestion de Base de Données Relationnelle (SGBDR). J'ai opté pour **MySQL**, un standard de l'industrie, reconnu pour sa fiabilité et sa rapidité d'exécution lors des requêtes de lecture (SELECT), ce qui est idéal pour un catalogue de formations.

En phase de développement, cette base de données est hébergée localement sur mon poste de travail via la solution **WAMP** (Windows, Apache, MySQL, PHP) et est administrée grâce à l'interface graphique **phpMyAdmin**.

La base de données principale du projet est la même base de données qu'on avait créé pour la mission 2:



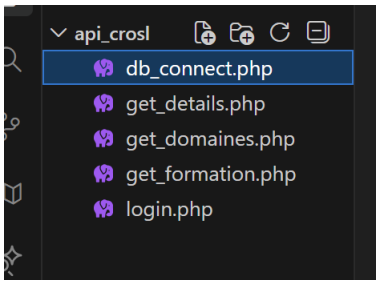
3. L'API Web (Le Serveur PHP)

3.1 Rôle et Architecture de l'API

Pour faire communiquer l'application mobile avec la base de données MySQL, j'ai développé une **API Web** (Interface de Programmation d'Application) en PHP 8.

J'ai choisi de créer une API Web car c'est une méthode simple, efficace et universelle. L'application Android n'a pas d'accès direct à la base de données. À la place, elle demande des informations au serveur via une simple URL (une requête HTTP). Le serveur PHP joue le rôle d'intermédiaire : il va chercher les données et répond à l'application avec du texte brut au format **JSON**.

Cette architecture permet de bien séparer le design (qui est géré sur le téléphone) de la logique et de la sécurité (qui sont gérées sur le serveur).



3.2 Sécurité et Connexion aux données (PDO)

La sécurité des données est un enjeu majeur du projet. Pour la connexion entre les scripts PHP et la base de données MySQL, j'ai utilisé l'extension native **PDO (PHP Data Objects)** au sein d'un fichier centralisé `db_connect.php`.

L'utilisation de PDO m'a permis de mettre en place des **requêtes préparées** dans l'ensemble de mes scripts. Cette technique sépare strictement le code SQL des données saisies, ce qui neutralise totalement les risques de failles par **injections SQL**. De plus, les erreurs de connexion sont gérées proprement grâce à des blocs `try/catch`.

```php
db_connect.php
C: > wamp64 > www > api_crosl > db_connect.php
<?php
// Informations de connexion à la base de données WAMP
$host = "127.0.0.1";
$dbname = "forma_db";
$username = "root"; // Par défaut sous WAMP
$password = "";     // Par défaut sous WAMP (vide)

try {
    // Création de la connexion
    $pdo = new PDO("mysql:host=$host;dbname=$dbname;charset=utf8", $username, $password);
    // On configure PDO pour qu'il affiche les erreurs si on se trompe
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {
    // Si la connexion échoue, on affiche un message d'erreur au format JSON
    die(json_encode(["erreur_connexion" => $e->getMessage()]));
}
?>
```

3.3 Traitement et Formatage : Exemple d'un Endpoint

L'API est composée de plusieurs points d'entrée (scripts), comme `get_domaines.php` ou `get_details.php`.

Prenons l'exemple du script `get_formation.php`. Ce script attend un paramètre précis dans l'URL (le domaine choisi par l'utilisateur).

1. Il récupère ce paramètre de manière sécurisée via la superglobale `$_GET`.

```php
// On récupère le domaine envoyé par l'application (ex: "Informatique")
$domaineChoisi = $_GET['domaine'] ?? '';
```

2. Il prépare et exécute la requête SQL correspondante.
3. Il utilise la fonction `fetchAll(PDO::FETCH_ASSOC)` pour récupérer les résultats sous forme de tableau.
4. Enfin, il convertit ce tableau en texte grâce à la fonction `json_encode()` et l'affiche à l'écran pour l'Android.

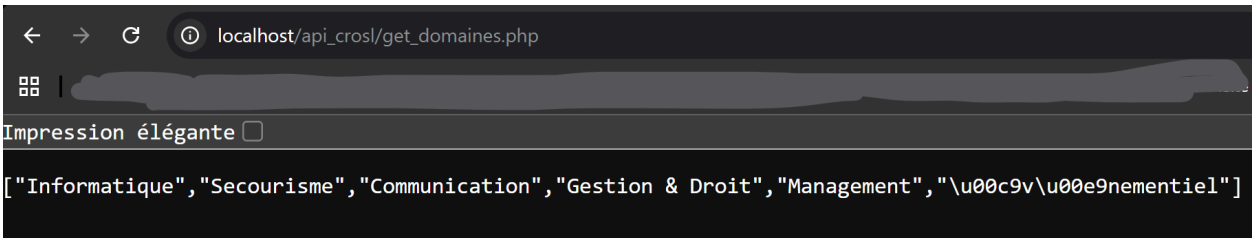
```php
try {
    // On prépare la requête pour chercher SEULEMENT les formations de ce domaine.
    // On récupère l'id, le titre et la date (ça nous servira pour la page 4 des détails)
    $requete = $pdo->prepare("SELECT id, titre, date_debut FROM formation WHERE domaine = :domaine");

    // On exécute la requête en remplaçant ":domaine" par le vrai domaine demandé
    $requete->execute(['domaine' => $domaineChoisi]);

    // On récupère toutes les lignes trouvées sous forme de tableau associatif
    $formations = $requete->fetchAll(PDO::FETCH_ASSOC);

    // On renvoie le résultat au format JSON
    echo json_encode($formations);
```

Pour m'assurer du bon fonctionnement de mon API indépendamment de l'application mobile, chaque script a été testé directement depuis un navigateur web. Le résultat renvoyé est bien un flux de données JSON brut.

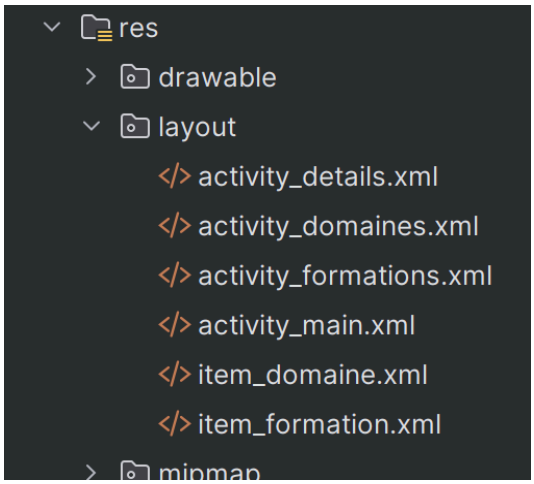


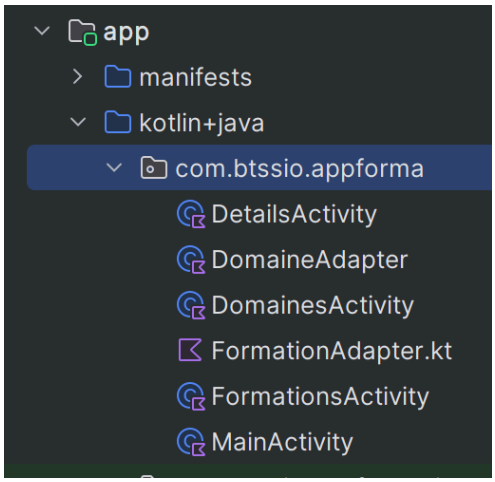
4. L'Application Android (Frontend)

4.1 Architecture du Projet et Interface Utilisateur (UI)

L'application mobile AppForma a été développée nativement sous Android Studio en utilisant le langage **Kotlin**. Pour garantir un code propre et maintenable, j'ai respecté le principe de séparation des préoccupations :

- **Le design (Vues) :** Les interfaces visuelles sont codées en **XML** (fichiers `layout`). J'ai veillé à créer des interfaces ergonomiques et "Responsive" pour s'adapter aux différentes tailles d'écrans.
- **La logique (Contrôleurs) :** Le code fonctionnel est écrit en **Kotlin** (fichiers `Activity` et `Adapter`). C'est ici que sont gérés les clics, les calculs et les appels réseau.





4.2 Communication Réseau et Asynchronisme

L'application doit télécharger les données depuis l'API Web. Sous Android, il est formellement interdit (et bloqué par le système) de faire une requête réseau sur le "Main Thread" (le fil d'exécution principal qui gère l'affichage), sous peine de figer l'écran de l'utilisateur (erreur ANR - *Application Not Responding*).

Pour contourner ce problème, j'ai implémenté un système d'**exécution asynchrone** (via un `Thread` en arrière-plan). L'application lance la requête HTTP vers mon serveur local (via l'alias `10.0.2.2` spécifique à l'émulateur Android). Pendant que le fichier JSON se télécharge en tâche de fond, l'utilisateur peut continuer à utiliser l'application fluidement. Une fois les données reçues, l'application repasse sur le Main Thread pour mettre à jour l'interface visuelle.

```
// 3. On lance la recherche des formations en arrière-plan
Thread {
    try {
        // On prépare l'URL avec le domaine en paramètre
        val adresse = "http://10.0.2.2/api_crosl/get_formation.php?domaine=$domaineChoisi"

        // On télécharge le résultat JSON
        val reponseJson = URL( spec = adresse).readText()
```

4.3 Traitement des données et Affichage Dynamique (RecyclerView)

Une fois le flux JSON réceptionné, l'application le désérialise (le traduit) pour le transformer en objets manipulables en Kotlin.

Pour afficher ces listes de données de manière optimisée (comme la liste des domaines ou des formations), j'ai utilisé le composant **`RecyclerView`**. Contrairement à une simple liste statique, le RecyclerView est un composant intelligent qui recycle les vues (les lignes de la liste) qui sortent de l'écran lors du défilement (scroll) pour économiser la mémoire du téléphone.

Pour le faire fonctionner, j'ai développé un **Adapter** personnalisé. Son rôle est de prendre les données téléchargées et de les injecter une par une dans un "moule" visuel XML défini à l'avance (comme le fichier `item_formation.xml`).

```
override fun onBindViewHolder(holder: FormationViewHolder, position: Int) {
    val formationActuelle = listeFormations[position]
    // On remplit les textes
    holder.tvTitre.text = formationActuelle.titre
    holder.tvDate.text = "Date de début : ${formationActuelle.date_debut}"
    // Quand on clique sur une formation
    holder.itemView.setOnClickListener {
        // On prépare le voyage vers la page des détails
        val intent = android.content.Intent( packageContext = holder.itemView.context,  cls = DetailsActivity::class.java)
        // On glisse l'ID de la formation dans les bagages (C'est un Int, un nombre entier)
        intent.putExtra( name = "ID_FORMATION",  value = formationActuelle.id)
        // On lance la nouvelle page
        holder.itemView.context.startActivity(intent)
    }
```

4.4 Navigation et Transmission de Données (Intents)

L'expérience utilisateur repose sur une navigation fluide entre les différents écrans (Activités). Lorsqu'un utilisateur clique sur un domaine (ex: "Secourisme"), l'application déclenche un **`Intent`** (une intention de navigation) pour ouvrir la page `FormationsActivity`.

Afin d'afficher uniquement les formations correspondantes, j'ai utilisé la méthode `putExtra()` de l'Intent. Cela permet "d'embarquer" le nom du domaine cliqué dans le voyage vers la nouvelle page. La nouvelle Activité récupère ce paramètre à son ouverture et s'en sert pour interroger l'API avec le bon domaine.

```
val intent = android.content.Intent( packageContext = holder.itemView.context,  cls = DetailsActivity::class.java)
// On glisse l'ID de la formation dans les bagages (C'est un Int, un nombre entier)
intent.putExtra( name = "ID_FORMATION",  value = formationActuelle.id)
```

5. Limites du projet et Axes d'amélioration

Bien que l'application réponde au cahier des charges initial du CROSL, les choix technologiques effectués pour ce prototype (maquette) présentent certaines limites qu'il conviendrait de corriger avant une véritable mise en production.

5.1 Dépendance totale au réseau (Pas de mode hors-ligne)

Le principal inconvénient de cette architecture Client-Serveur pure est sa dépendance à la connexion internet. Actuellement, si l'utilisateur ouvre l'application dans une zone sans réseau (ex: dans le métro ou une salle de sport isolée), l'application ne pourra rien afficher car aucune donnée n'est sauvegardée sur le téléphone.

- **L'axe d'amélioration :** Mettre en place une architecture **"Offline-First"**. Il faudrait utiliser une base de données locale (comme la bibliothèque **Room** / SQLite) intégrée au téléphone. L'application téléchargerait les données PHP une seule fois et les stockerait localement.

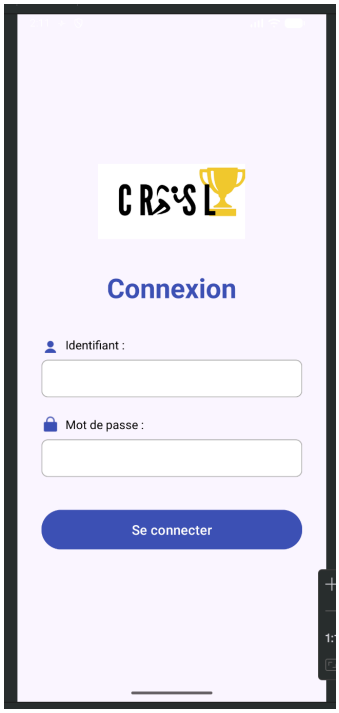
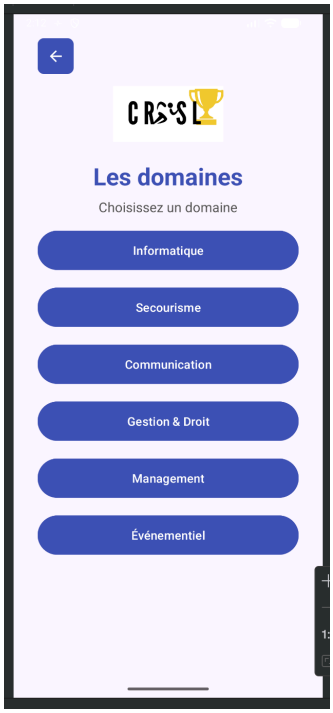
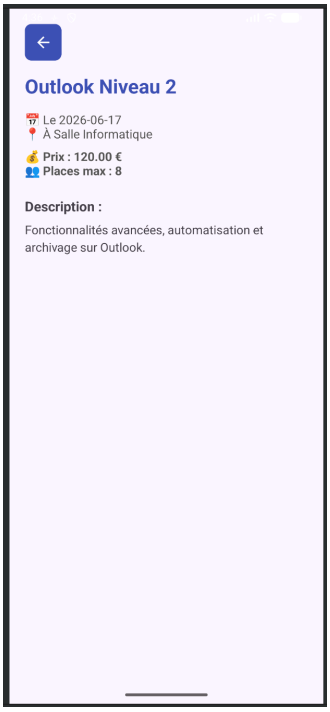
5.2 Sécurité des communications (HTTP)

Dans ce projet de développement local, les requêtes transitent via le protocole HTTP. Cela signifie que les données voyagent "en clair" sur le réseau, ce qui expose l'application à des attaques de type **"Homme du Milieu"** (Man-in-the-Middle). De plus, l'API est publique et ne vérifie pas l'identité de l'application qui l'interroge.

- **L'axe d'amélioration :** Pour une mise en production, il serait impératif d'héberger l'API sur un serveur disposant d'un certificat SSL/TLS pour chiffrer les échanges en **HTTPS**.

Conclusion

Le projet **AppForma** m'a permis de concevoir et de déployer une architecture Client-Serveur complète (Android, PHP, MySQL). Bien que ce prototype nécessite quelques optimisations pour une mise en production (sécurité HTTPS, mode hors-ligne), il répond parfaitement au besoin du CROSL : offrir un catalogue de formations mobile, dynamique et fonctionnel. Ce projet concrétise l'ensemble de mes compétences techniques et me prépare efficacement aux exigences du monde professionnel

Annexe

1. Aperçu de l'Interface Utilisateur